

프로그래밍 역량 강화 전문기관, 민코딩

SSD 프로젝트 기능 추가 요청



목차

1. 기능 추가 미션 소개
2. SSD – Erase 기능 추가
3. Logger 기능 추가
4. Runner 기능 추가
5. SSD - Command Buffer 기능 추가
6. 기능 추가 미션, 유의사항

기능 추가 미션 소개

기능 추가, 리팩토링 미션 개요

[참고] 팀 프로젝트 일정

	팀 프로젝트								
08:30 – 09:30	수평적 코드 리뷰를 위한 팀 빌딩 - 커뮤니케이션	Team Project 미션1 수행	Team Project 미션 2 안내 Team Project 미션 2 안내	Team Project 미션 2 수행	Team Project 마무리 및 발표준비				
09:30 – 10:30									
10:30 – 11:30									
11:30 – 12:30									
12:30 – 13:30	Team Project 개발환경 세팅 Team Project 미션1 안내	Team Project 미션1 수행	Team Project 미션 2 수행	Team Project 미션 2 수행	Team Project 발표				
13:30 – 14:30									
14:30 – 15:30	Team Project 미션1 수행								
15:30 – 16:30									
16:30 – 17:30	사내 현업활동 가이드								

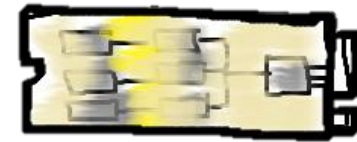
3 ~ 4일차 미션

기능 확장을 염두에 두고 클린하게 리팩토링한 구조 위에,
실제 기능을 추가해보자

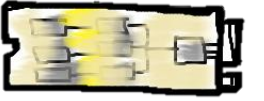
미션

- SSD - Erase 명령어 추가
- Logger 기능 추가
- Runner 기능 추가
- SSD - Command Buffer 기능 추가

SSD – Erase 기능 추가



Erase 기능이 지원되도록, 가상 SSD와 Shell 에 명령어를 추가한다.



SSD - Erase 기능 추가

Write 명령어 처럼 **SSD 자체에 Erase 기능을 추가**하고,
Shell에서 명령어를 수행할 수 있도록 한다.

- SSD 명령어 : E [LBA] [SIZE]
 - 특정 LBA 부터 특정 Size 까지 내용을 삭제한다.
 - **삭제할 수 있는 최대 Size는 최대 10칸이다.**
 - SSD에서 삭제하면 0x00000000이 된다.
 - 범위에 벗어난 경우, **ssd_output.txt**에 **ERROR**를 기록한다.
- Shell 명령어 1 : erase [LBA] [SIZE]
 - SSD에 명령어를 수행한다.
- Shell 명령어 2 : erase_range [Start LBA] [End LBA]
 - Start LBA 부터 End LBA 까지 내용을 삭제한다.

```
OneWRtest()  
  SSD_WRITE(0, 0xAB)  
  a = SSD_READ(0)  
  if (a == 0) print("PASS")  
  else print("FAIL")
```

Erase, Test Script 추가하기

Write Read 반복 테스트

- Test Script 이름 : 4_EraseAndWriteAging

Test Scenario

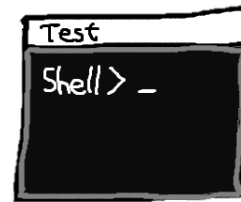
- 0 ~ 2번 LBA 삭제
- Loop 30회
 - 2번 LBA Write
 - 2번 LBA OverWrite
 - 2 ~ 4번 LBA 삭제
 - 4번 LBA Write
 - 4번 LBA OverWrite
 - 4 ~ 6번 LBA 삭제
 - 6번 LBA Write
 - 6번 LBA OverWrite
 - 6 ~ 8번 LBA 삭제



전체 LBA 영역에 대해
같은 규칙으로 수행한다.
여기까지가 1 Cycle 이며 30 Cycle까지 해야한다.

Logger 기능 추가

Test Shell 이 출력하는 Log를 관리해주는 시스템 제작



Logger 가 필요한 이유

테스트를 진행하면서 Fail이 발생하면 Log Message를 불량 분석을 하게 된다.

검증팀의 여러 사람들이 각자 테스트 시나리오를 작성하다 보면,

통일되지 않은 로그 포맷을 사용하지 않아, 불량 분석에 시간이 걸리기도 하고

로그들을 Text파일로 저장하는데 로그 데이터가 대량으로 쌓이면서 파일 관리가 어렵다.

위와 같은 문제를

해결해주는 기능을 추가하자.

우리 팀만의 print 함수를 만들면 된다.

-> 화면 출력이 되면서, 동시에 파일 Write가 되는 print 함수

Logger 기능 1 : 통일된 로그

로그 포맷

- 출력 포맷 : [날짜 시간] 클래스명.함수명() : 로그메세지
- 함수명이 들어가는 공간에는 **최소 30칸**을 확보하여 함수명을 기입한다.

로그 포맷 예시

```
[26.01.01 17:04] Sender.sendMessageWithData( ) : send and wait sync message.  
[26.01.01 17:04] Message.checkMessage( ) : FAIL - Response message didn't arrive.  
[26.01.01 17:04] Shell.release( ) : Bye bye.
```



`logger.print("Shell.release()", "Bye bye.")` 호출시

Logger 기능 2 : 로그파일 관리

실제로 대부분의 불량 분석은 Last log 10MB 로 해결된다.
하지만 하나의 파일이 TB 이상이 된 상태로 불량분석에 시간이 오래걸리게 된다.
우리의 프로젝트에서는 조금 더 사이즈를 축소하여 10MB가 아닌
10KB 단위로 파일을 관리해보자.



until_260307_17h_12m_1
1s.log

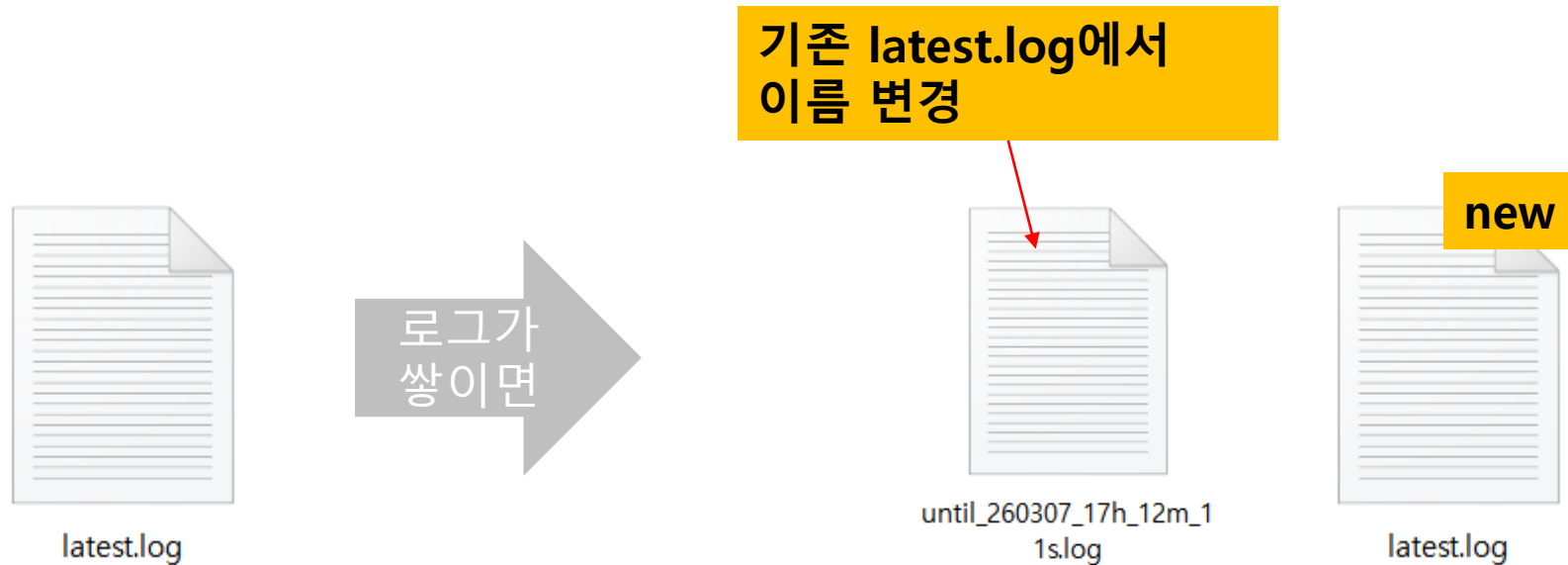


until_260307_17h_45m_3
2s.log

Logger 기능 2 : 로그 기록 규칙

로그 기록 규칙

1. 로그를 남기기 위해 latest.log 파일을 생성한다.
2. 10kb 까지 기록을 수행한다.
3. 10kb가 넘으면 해당 파일의 이름을 "until_날짜_시간.log" 으로 변경한다.



파일명 : 마지막 로그 기록의 날짜 + 시간

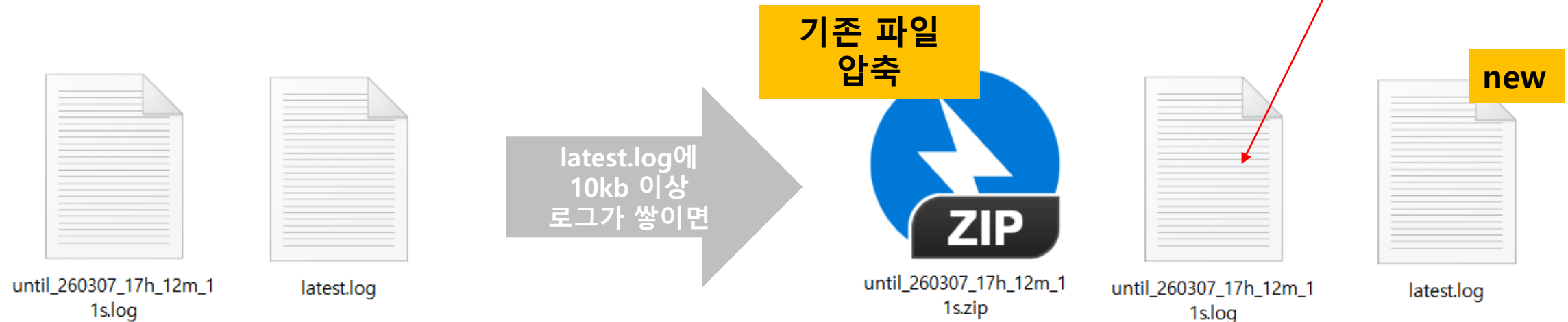
Logger 기능 2 : 로그 압축 규칙

로그 압축 규칙

1. 두 개의 until_날짜_시간.log 파일이 생성되면,
가장 이전에 생성된 log파일을 압축한다.

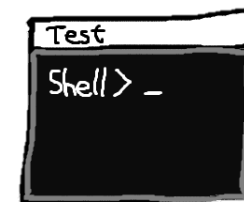
1. 압축 Library를 썼다고 가정하고, 파일명만 .log -> .zip 으로 변경한다.

기존 latest.log에서
이름 변경



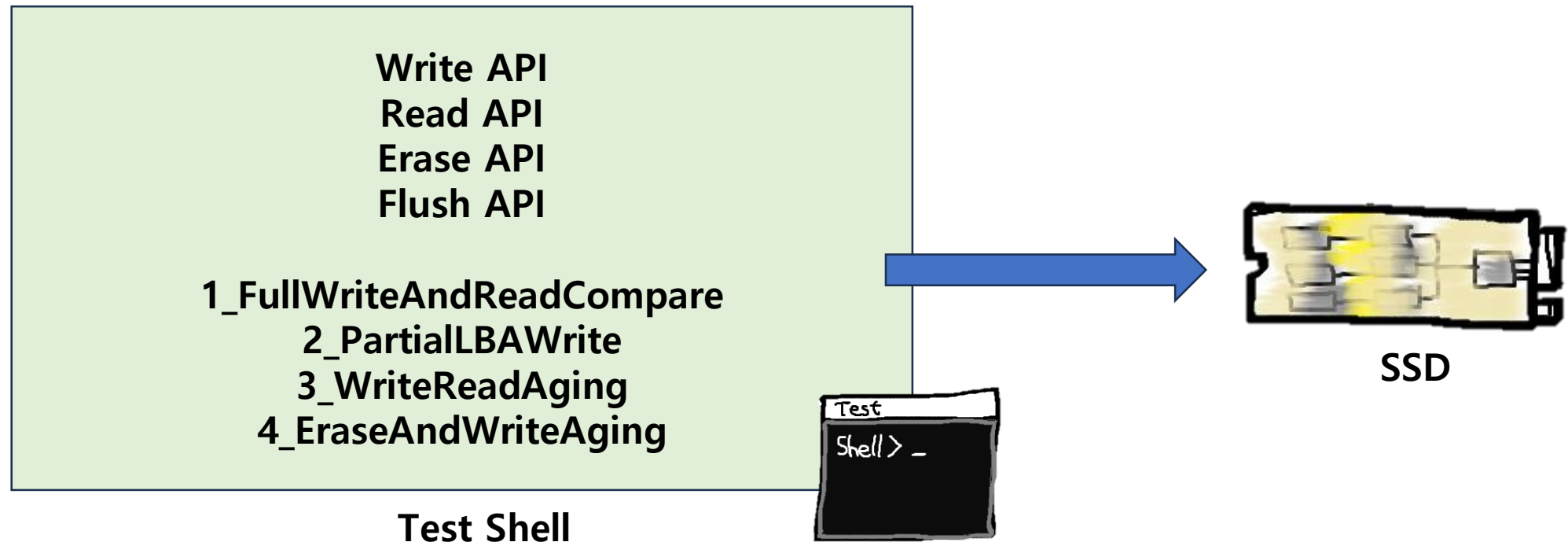
Runner 기능 추가

Test Script의 이름들을 목록으로 만들어, 순차적으로 Test를 수행하는 기능



현재 구조

기본 Command와 Test Script들이 만들어져있다.



Runner

[현재상황]

테스트 Script들은 하나의 함수 형태로 만들어져 있으며,
Shell을 실행시켜, 개별 Test Script를 수행할 수 있다.

[요구사항]

검증 팀에서는 여러개의 Test Script를 실행시키기 위해,
하나의 Test Script가 끝날때 까지 기다리고 있을 수 없다.

먼저 Test를 수행하고자 하는 Test Script의 목록을 txt파일로 저장한다.

Test Shell을 실행시킬 때 txt파일과 함께 입력하면,

txt파일에 기입된 Test Script들을 순차적으로 실행하는 기능을 추가하고자 한다.

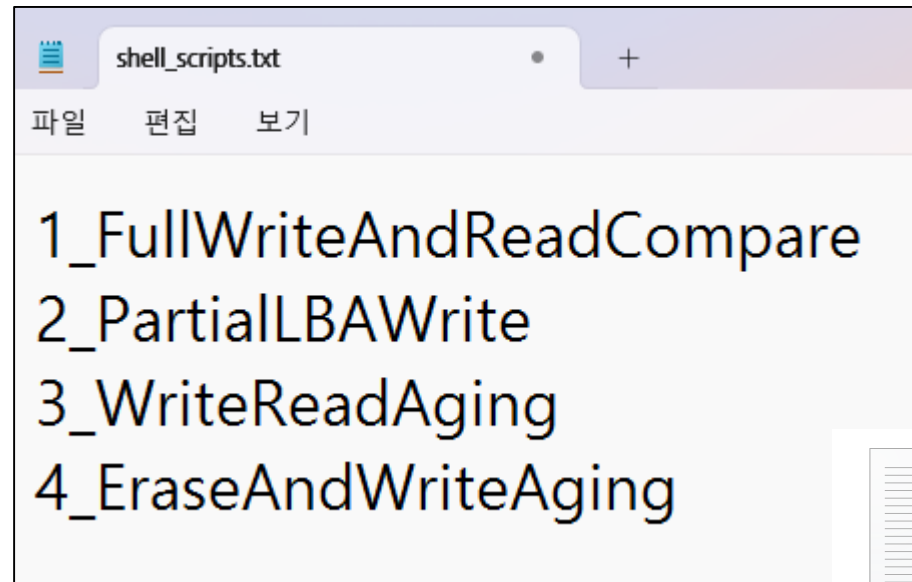
예시

shell_scripts.txt 파일에 실행시키고 싶은 Test Script 목록을 기입한다.
그리고 명령프롬프트에서

```
C:\Users\minco>shell shell_scripts.txt
```

Test Shell 실행파일 이름 예시

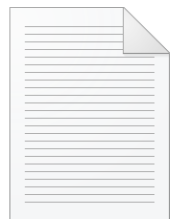
이라고 수행하면
테스트가 하나씩 수행된다.



```
shell_scripts.txt
파일 편집 보기

1_FullWriteAndReadCompare
2_PartialLBWrite
3_WriteReadAging
4_EraseAndWriteAging
```

Test Script 기입 예시



shell_scripts

결과 화면

1 화면에는 Test가 수행중임을 알리는 Text를 출력한다.
동작Text는 우측 예시를 참고
스크립트 이름 --- Run ... 이 먼저 출력된다.

2 테스트가 Pass로 종료되면 "Pass" 메시지 출력 후,
다음 스크립트가 수행된다.

3 테스트가 Fail로 종료되면 "FAIL!" 메시지 출력 후
즉시 프로그램이 종료된다.
더이상 스크립트를 수행하지 않는다.

1

```
1_FullWriteAndReadCompare    ___    Run...Pass
2_PartialLBWrite             ___    Run...
```



2

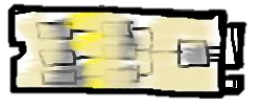
```
1_FullWriteAndReadCompare    ___    Run...Pass
2_PartialLBWrite             ___    Run...Pass
3_WriteReadAging            ___    Run...
```



3

```
1_FullWriteAndReadCompare    ___    Run...Pass
2_PartialLBWrite             ___    Run...Pass
3_WriteReadAging            ___    Run...FAIL!
```

SSD - Command Buffer 기능 추가



Command Buffer 및 Flush 명령어 기능 추가

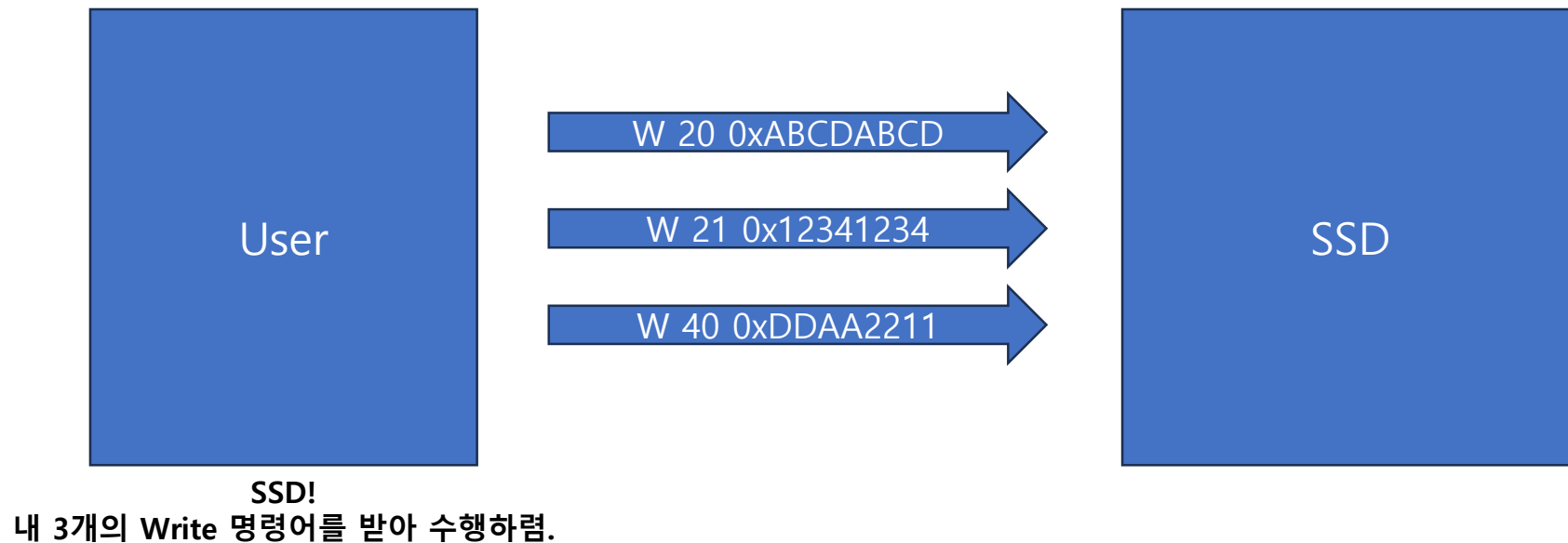
Command Buffer 를 활용한 최적화

Command Buffer의 도입 목적

- 불필요한 Command 개수를 줄여, 성능을 향상
- Command들을 즉시 처리하지 않고, 모아둠으로써
수행을 할 필요가 없는 명령어를 찾아내어 제거한다.

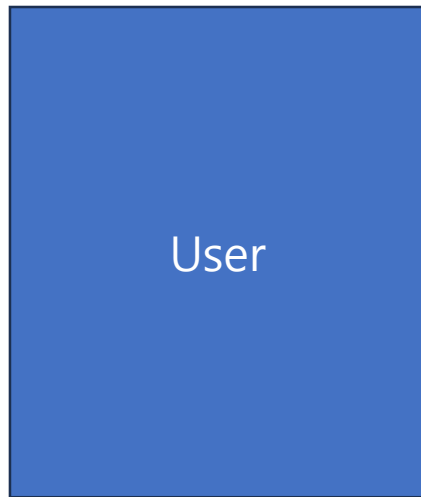
Command Buffer 개념 1

User가 가상 SSD에 3회의 Write를 수행한다고 가정한다.

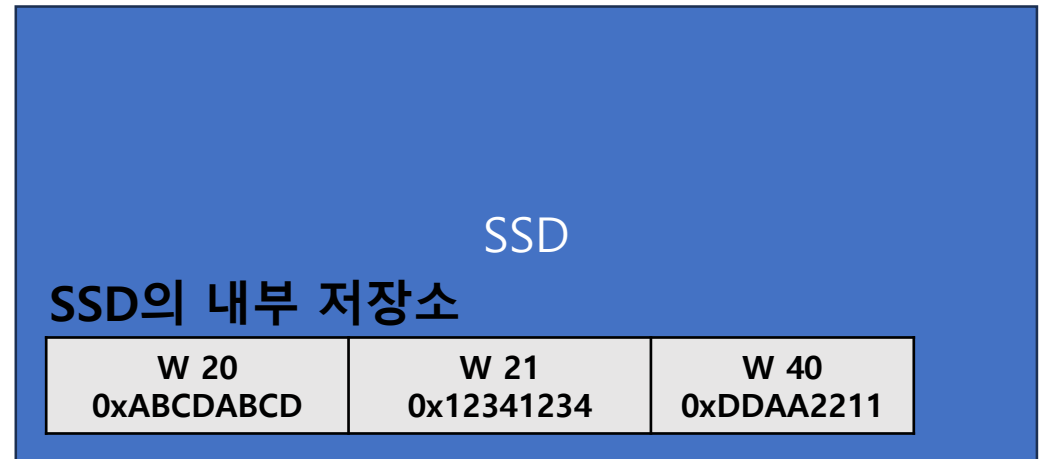


Command Buffer 개념 2

SSD는 내부 저장소를 가지고 있어, Command들을 보관해둔다.
처리 속도를 더 높이는 알고리즘을 적용하기 위해,
한꺼번에 모아서 처리하려는 목적이다.



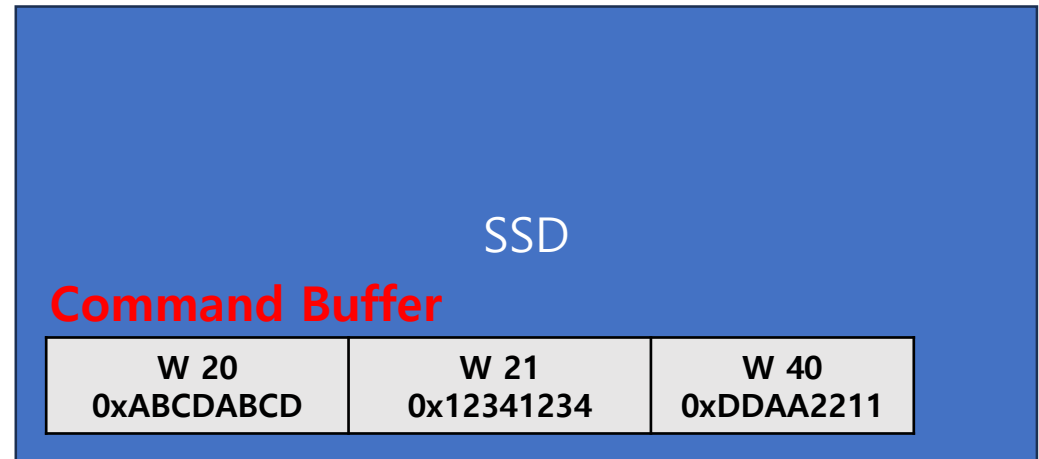
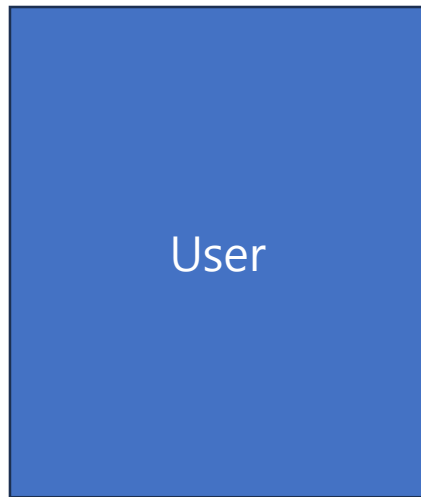
SSD는
내 명령을 다 수행했는지?



User의 명령들은
일단 보관했다가 한꺼번에 처리해야지.

Command Buffer 개념 3

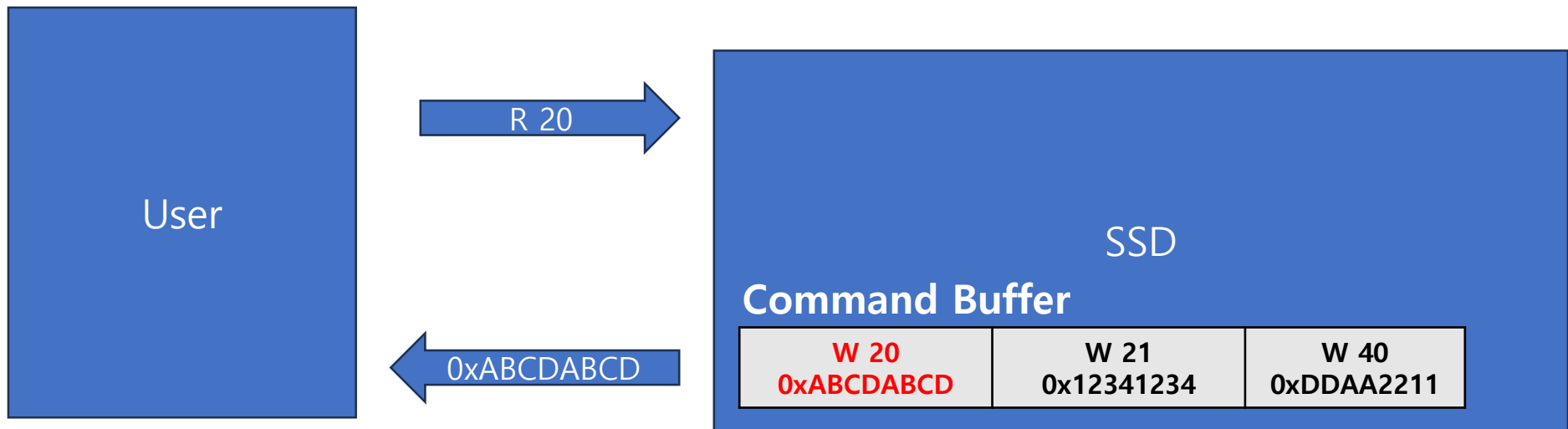
SSD의 내부 저장소를 **Command Buffer** 라고 한다.



Command Buffer 개념 4

이 상황에서 User가 Read 20 LBA를 요청하면
nand를 확인하지 않고, buffer에 있는 값을 찾아 리턴해야한다.

- nand 보다 buffer에 있는 내용이 더 최신 정보이기 때문



Flush 명령어 추가 (SSD, Test Shell)

SSD – **Flush** 기능 (명령어 추가 필요)

- Command Buffer에 있는 모든 명령어들을 수행하여 Buffer 전체 비운다.
- SSD 명령어 이름 : **F**
- Test Shell 명령어 이름 : **flush**

SSD – Command Buffer 세부사항

Command Buffer 세부사항

- Write와 Erase명령어만 Buffer에 저장한 후, 한꺼번에 명령어들을 수행한다.
그 외 명령어는 Buffer에 담지 않는다.
- Buffer에는 **최대 5개**의 명령어를 기록할 수 있다.
- Buffer가 꽉찬 상태에서 새로운 명령어가 들어오면,
즉시 Flush 수행 후, 새로운 명령어를 Buffering한다.

SSD – Command Buffer 구현 방법 1

buffer 폴더를 생성한다.

SSD 프로그램을 시작했고, 만약 buffer 폴더가 없다면 buffer 폴더 생성과 5개의 기본 파일을 만들어둔다.

- 1_empty
- 2_empty
- 3_empty
- 4_empty
- 5_empty



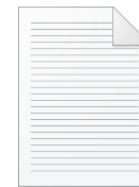
buffer



1_empty



2_empty



3_empty



4_empty



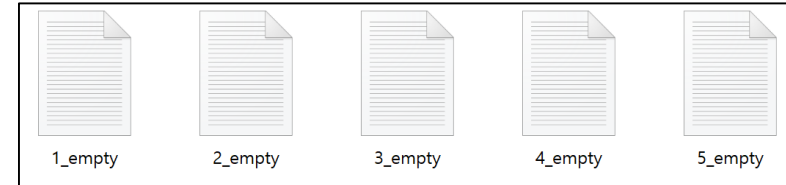
5_empty

SSD – Command Buffer 구현 방법 2

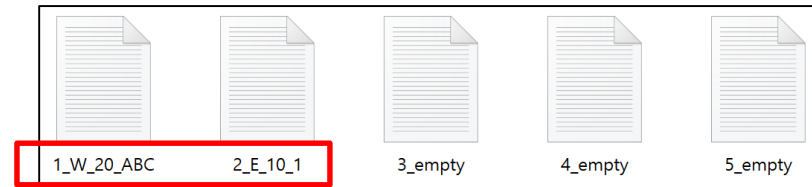
버퍼에 어떤 값들이 채워져있는지 확인 방법
buffer 폴더 내부 파일 목록을 가져온다.

버퍼에 값 기록 방법
명령어가 들어올 때 마다, 파일명을 변경한다.

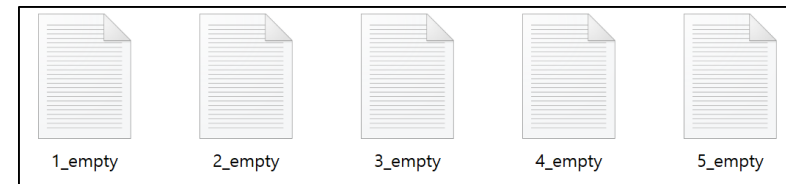
버퍼 비우는 방법
buffer 폴더 내 모든 파일명을 변경한다.



2개의 명령어가
순차적으로 들어온 후



Flush 수행



Command Buffer 알고리즘 1 - Ignore Command

Ignore Command 알고리즘

- 같은 LBA에 Write하는 명령어가 여러개 들어왔다면,
마지막 유효한 명령어만 남겨두고, 기존 명령어는 Buffer에서 삭제한다.

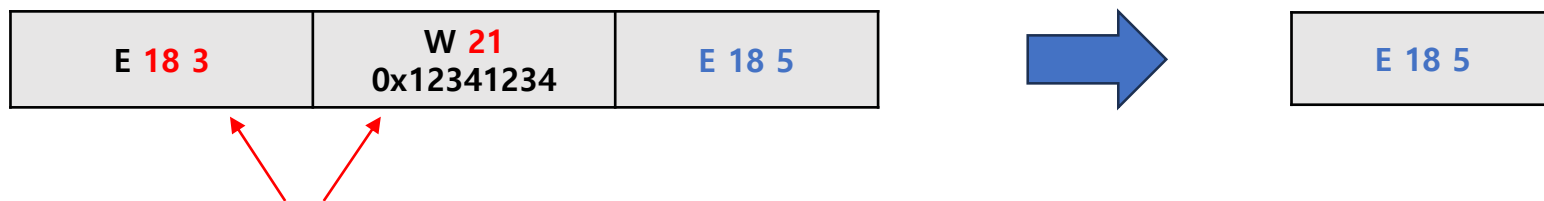


LBA 20번에는 Overwrite가 될 예정이므로
수행할 필요가 없는 명령어이다.

Command Buffer 알고리즘 1 - Ignore Command

Ignore Command 알고리즘

- Erase 명령어를 수행하기 이전, 같은 LBA를 Write하거나 Erase하는 명령어가 존재한다면 명령을 수행할 필요가 없기에 Buffer에서 제거한다.

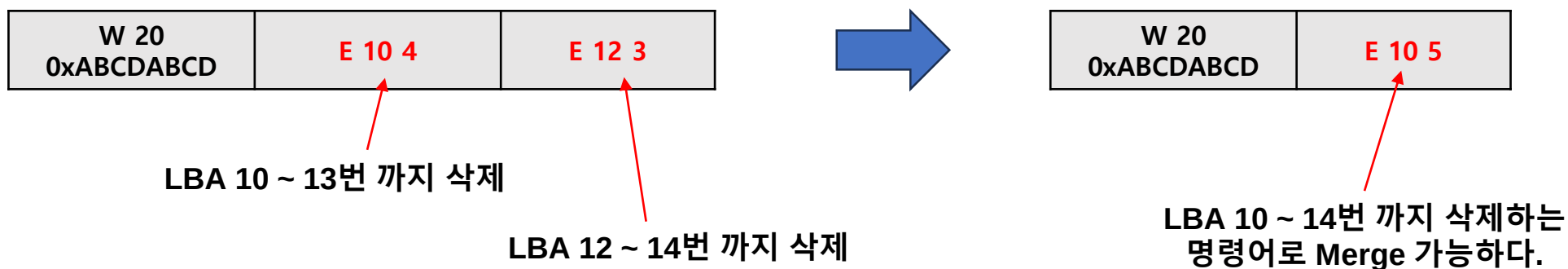


LBA 18 ~ 22번 까지 삭제 될 예정이므로,
수행할 필요가 없는 명령어이다.

Command Buffer 알고리즘 2 - Merge Erase

Merge Erase 알고리즘

- Erase 명령어가 연속적으로 2회 들어왔고,
2개의 Erase 명령어가 연속적인 LBA Range를 가질 수 있다면
두 명령어를 하나의 명령어로 교체한다.
- Erase 명령어의 Size는 10보다 클 수 없음에 유의한다.



Command Buffer 알고리즘 3 - Fast Read

Fast Read

- Read를 수행할 때, Command Buffer 부터 확인을 한다.
- Buffer에 Read할 값이 적혀 있다면,
실제 NAND를 읽지 않고도, Buffer로 부터 값을 찾아 값을 반환할 수 있다.

W 10 0xABCDABCD	E 10 2	W 11 0xABCDABCD
--------------------	--------	--------------------

위 상황에서 **R 10** 명령어가 주어진다면, Nand를 읽지 않고 **0x00000000**이 읽혀야한다.

위 상황에서 **R 11** 명령어가 주어진다면, Nand를 읽지 않고 **0xABCDABCD**가 읽혀야한다.

기능 추가 미션, 유의사항

미션 수행시, 유의사항

유의사항

1. 3일차 부터는 TDD로 개발하지 않아도 됩니다.
2. 일부러 지저분하게 작성 후 Commit을 해주어도 좋습니다.
개발과 리팩토링을 동시에 하는 것을 자제해주세요.
3. 개발 후 동작이 잘 되면 Commit을 하고,
클린코드가 되게 꼼꼼 리팩토링을 하여 Commit을 해주세요.
전 후 비교되게끔 해주시면, 발표자료를 만들 때 사용하기 좋습니다.

디자인 패턴을 사용시, 추가 점수

디자인 패턴은 여러 곳에서 발생할 수 있는 문제를 해결하는 일반화된 모범 사례입니다. 이를 클린코드의 Best Practice 라고 합니다. 이로 인해, GoF 디자인패턴을 사용하면 약간의 추가 점수가 주어집니다.

예시)

- Singleton Pattern
- Command Pattern 등